

Práctica 4-8 - Estado de implementación – 06/10/2026

La práctica se desarrollará de forma incremental siguiendo un enfoque Bottom-Up: primero se implementan y prueban las clases más básicas e independientes, y después se construyen las clases que dependen de ellas hasta integrar el sistema completo.

Implementado hasta el momento:

- **Persona.java:** Clase abstracta base del sistema. Incluye los atributos protegidos *id*, *nombre* y *correo*; constructor vacío y constructor con datos; getters y setters; y el método abstracto *mostrarInformacion()*.
- **Usuario.java:** Hereda de Persona. Incluye *librosPrestados*, *telefonoCelular* y *activo*; varios constructores para practicar sobrecarga; getters y setters; *solicitarPrestamo()* y *devolverLibro()*; y la sobreescritura de *mostrarInformacion()*.
- **Bibliotecario.java:** Hereda de Persona. Incluye *turno* y *salario*; constructor vacío y constructor completo; getters y setters; *autorizarPrestamo()*, *registrarLibro()* y *eliminarLibro()*; y la sobreescritura de *mostrarInformacion()*. Por ahora, los tres métodos de gestión son provisionales y sólo muestran mensajes, ya que todavía no existen las clases con las que deberán interactuar.
- **Main.java:** Realiza una prueba inicial creando un Usuario y un Bibliotecario mediante referencias de tipo Persona. Permite comprobar herencia, sobreescritura y polimorfismo mediante *mostrarInformacion()*.

Pendiente por implementar:

- **Libro.java:** Atributos *isbn*, *titulo*, *autor*, *anioPublicacion* y *disponible*; atributo static *totalLibros*; constructores solicitados; y métodos *estaDisponible()*, *prestar()* y *devolver()*.
- **Prestamo.java:** Relacionará un Usuario con un Libro. Debe almacenar *usuario*, *libro*, *fechaPrestamo*, *fechaDevolucion* y *activo*; además de *iniciarPrestamo()*, *finalizarPrestamo()* y *estaActivo()*.
- **Biblioteca.java:** Será la clase que integre el sistema mediante ArrayList de libros, usuarios, bibliotecarios y préstamos. Debe incluir altas, búsquedas, eliminación, catálogo, usuarios, préstamos y devoluciones. También se trabajará la sobrecarga de *buscarLibro()*.

Integración de Bibliotecario: Cuando existan Libro y Biblioteca, se revisarán los métodos *autorizarPrestamo()*, *registrarLibro()* y *eliminarLibro()* para que dejen de ser únicamente demostrativos y participen en operaciones reales del sistema.

Main final y casos de prueba: Se deberá ampliar para registrar los libros y usuarios solicitados, realizar un préstamo, comprobar la disponibilidad del libro, realizar la devolución y verificar el estado final.

Interfaz Prestable: Es un punto extra. Sólo se implementará si se decide incluirlo; en ese caso Libro deberá implementar los métodos prestar() y devolver() definidos por la interfaz.

Orden de construcción propuesto - Bottom-Up

- 1 Persona
- 2 Usuario y Bibliotecario
- 3 Libro
- 4 Prestamo
- 5 Biblioteca
- 6 Main con integración completa y casos de prueba
- 7 Prestable, únicamente si se realiza el punto extra

Observación

Los cuatro archivos actuales constituyen la base del proyecto, no el sistema completo. La intención es conservar versiones funcionales por etapas y subir los avances a Git conforme se incorporen las clases que dependen de las anteriores.

Código Persona.java

```
1  /*
2  Programación Orientada a Objetos - FI UNAM - Semestre 2027-1
3  Copado Hernández Luis Felipe
4  De La Cruz García Juan Antonio
5  Rivera Hernández Luis Angel
6  Sandoval Casimiro Diego Enrique
7  Código 3: Define la clase Bibliotecario, que hereda de Persona y agrega
8  los atributos turno y salario, además de operaciones propias del empleado.
9  */
10 public class Bibliotecario extends Persona { 1 uso
11     private String turno; 5 usos
12     private double salario; 5 usos
13
14     // Constructor vacío
15     public Bibliotecario() { 0 usos
16         super();
17         this.turno = "";
18         this.salario = 0.0;
19     }
20
21     // Constructor completo
22     public Bibliotecario(int id, String nombre, String correo, String turno, double salario) { 1 uso
23         super(id, nombre, correo);
24         this.turno = turno;
25         this.salario = salario;
26     }
27
28     // Métodos setters
29     > public void setTurno(String turno) { this.turno = turno; }
32
33     > public void setSalario(double salario) { this.salario = salario; }
36
37     // Métodos getters
38     > public String getTurno() { return turno; }
41
42     > public double getSalario() { return salario; }
45
46     // Métodos indicados para el bibliotecario
47     > public void autorizarPrestamo() { System.out.println("Préstamo autorizado."); }
50
51     > public void registrarLibro() { System.out.println("Libro registrado."); }
54
55     > public void eliminarLibro() { System.out.println("Libro eliminado."); }
58
59     // Sobreescritura del método abstracto definido en Persona
60     @Override 2 usos
61     @+ public void mostrarInformacion() {
62         System.out.println("ID: " + id);
63         System.out.println("Nombre: " + nombre);
64         System.out.println("Correo: " + correo);
65         System.out.println("Turno del empleado: " + turno);
66         System.out.println("Salario del empleado: " + salario);
67     }
68 }
```

Código Usuario.java

```
1  /*
2  Programación Orientada a Objetos - FI UNAM - Semestre 2027-1
3  Copado Hernández Luis Felipe
4  De La Cruz García Juan Antonio
5  Rivera Hernández Luis Ángel
6  Sandoval Casimiro Diego Enrique
7  Código 2: Define la clase Usuario, que hereda de Persona y agrega información
8  relacionada con los préstamos y el teléfono celular del usuario.
9  */
10 public class Usuario extends Persona { 1 uso
11     private int librosPrestados; 10 usos
12     private String telefonoCelular; 7 usos
13     private boolean activo; 7 usos
14
15     // Constructor vacío
16     public Usuario() { 0 usos
17         super();
18         this.librosPrestados = 0;
19         this.telefonoCelular = "";
20         this.activo = true;
21     }
22
23     // Constructor solicitado para trabajar el estado del usuario
24     public Usuario(int id, String nombre, String correo, boolean activo) { 0 usos
25         super(id, nombre, correo);
26         this.librosPrestados = 0;
27         this.telefonoCelular = "";
28         this.activo = activo;
29     }
30
31     // Constructor con teléfono celular
32     public Usuario(int id, String nombre, String correo, String telefonoCelular) { 0 usos
33         super(id, nombre, correo);
34         this.librosPrestados = 0;
35         this.telefonoCelular = telefonoCelular;
36         this.activo = true;
37     }
38
39     // Constructor completo
40     public Usuario(int id, String nombre, String correo, String telefonoCelular, boolean activo) { 1 uso
41         super(id, nombre, correo);
42         this.librosPrestados = 0;
43         this.telefonoCelular = telefonoCelular;
44         this.activo = activo;
45     }
46
47     // Métodos setters
48 > public void setLibrosPrestados(int librosPrestados) { this.librosPrestados = librosPrestados; }
49
50 > public void setTelefonoCelular(String telefonoCelular) { this.telefonoCelular = telefonoCelular; }
51
52 > public void setActivo(boolean activo) { this.activo = activo; }
53
54     // Métodos getters
55 > public int getLibrosPrestados() { return librosPrestados; }
56
57 > public String getTelefonoCelular() { return telefonoCelular; }
58
59 > public boolean isActivo() { return activo; }
```

```

72
73 // Aumenta la cantidad de libros prestados al usuario
74 public void solicitarPrestamo() { librosPrestados++; }
75
76 // Disminuye la cantidad de libros prestados sin permitir valores negativos
77 public void devolverLibro() {
78     if (librosPrestados > 0) {
79         librosPrestados--;
80     }
81 }
82
83 // Sobreescritura del método abstracto definido en Persona
84 @Override
85 public void mostrarInformacion() {
86     System.out.println("ID: " + id);
87     System.out.println("Nombre: " + nombre);
88     System.out.println("Correo: " + correo);
89     System.out.println("Teléfono celular: " + telefonoCelular);
90     System.out.println("Número de libros prestados: " + librosPrestados);
91     System.out.println("Usuario activo: " + activo);
92 }
93
94 }
95

```

Código Bibliotecario.java

```
1  /*
2  Programación Orientada a Objetos - FI UNAM - Semestre 2027-1
3  Copado Hernández Luis Felipe
4  De La Cruz García Juan Antonio
5  Rivera Hernández Luis Ángel
6  Sandoval Casimiro Diego Enrique
7  Código 3: Define la clase Bibliotecario, que hereda de Persona y agrega
8  los atributos turno y salario, además de operaciones propias del empleado.
9  */
10 public class Bibliotecario extends Persona { 1 uso
11     private String turno; 5 usos
12     private double salario; 5 usos
13
14     // Constructor vacío
15     public Bibliotecario() { 0 usos
16         super();
17         this.turno = "";
18         this.salario = 0.0;
19     }
20
21     // Constructor completo
22     public Bibliotecario(int id, String nombre, String correo, String turno, double salario) {
23         super(id, nombre, correo);
24         this.turno = turno;
25         this.salario = salario;
26     }
27
28     // Métodos setters
29     public void setTurno(String turno) { this.turno = turno; }
30
31
32     public void setSalario(double salario) { this.salario = salario; }
33
34
35
36
37     // Métodos getters
38     public String getTurno() { return turno; }
39
40
41     public double getSalario() { return salario; }
42
43
44
45
46     // Métodos indicados para el bibliotecario
47     public void autorizarPrestamo() { System.out.println("Préstamo autorizado."); }
48
49
50     public void registrarLibro() { System.out.println("Libro registrado."); }
51
52
53     public void eliminarLibro() { System.out.println("Libro eliminado."); }
54
55
56
57
58     // Sobreescritura del método abstracto definido en Persona
59     @Override 2 usos
60     public void mostrarInformacion() {
61         System.out.println("ID: " + id);
62         System.out.println("Nombre: " + nombre);
63         System.out.println("Correo: " + correo);
64         System.out.println("Turno del empleado: " + turno);
65         System.out.println("Salario del empleado: " + salario);
66     }
67 }
68 }
```

Código Main.java

```
1  /*
2  Programación Orientada a Objetos - FI UNAM - Semestre 2027-1
3  Copado Hernández Luis Felipe
4  De La Cruz García Juan Antonio
5  Rivera Hernández Luis Ángel
6  Sandoval Casimiro Diego Enrique
7  Código 4: Crea objetos Usuario y Bibliotecario y comprueba la herencia,
8  la sobrescritura y el polimorfismo mediante el método mostrarInformacion().
9  */
10 public class Main {
11     public static void main(String[] args) {
12         // Las referencias son de tipo Persona y los objetos son de sus clases hijas.
13         Persona usuario1 = new Usuario(
14             id: 1,
15             nombre: "Juan Perez",
16             correo: "juan@gmail.com",
17             telefonoCelular: "55-12-34-56-78",
18             activo: true
19         );
20
21         Persona bibliot1 = new Bibliotecario(
22             id: 101,
23             nombre: "Ana Gomez",
24             correo: "ana@biblioteca.com",
25             turno: "Matutino",
26             salario: 15000.0
27         );
28
29         System.out.println("PRUEBA DE CLASES");
30         usuario1.mostrarInformacion();
31         System.out.println("-----");
32         bibliot1.mostrarInformacion();
33     }
34 }
```